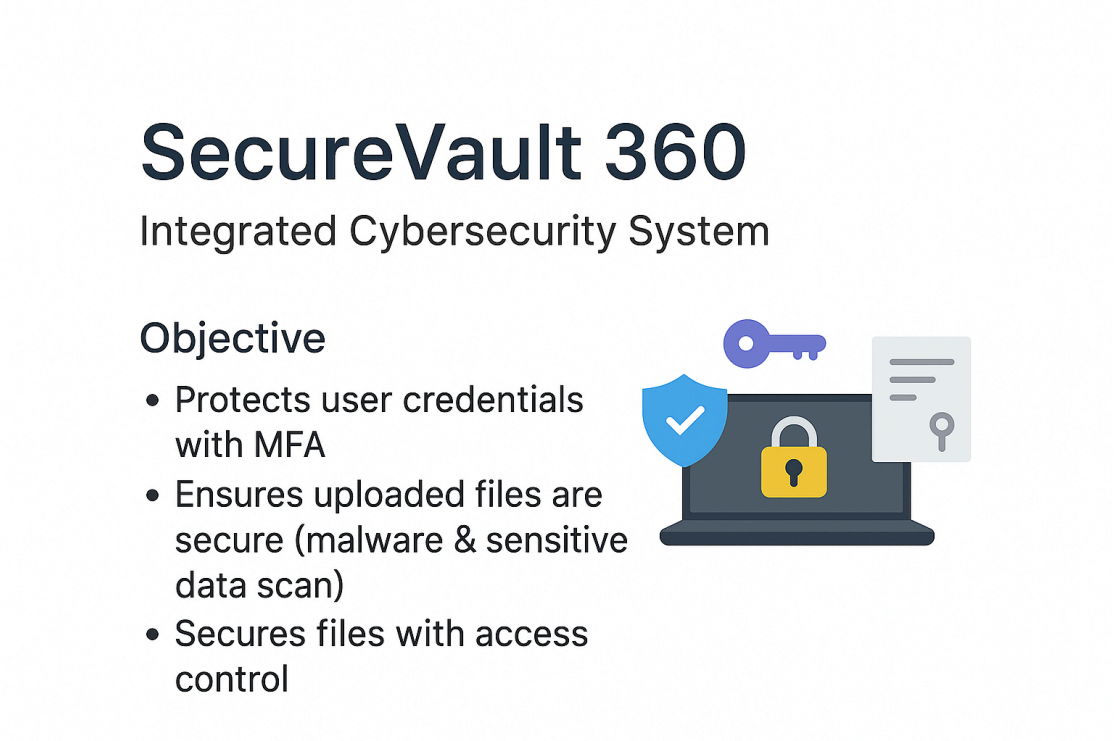
PBL SEM 5 : ⚡ **SecureVault 360** = Protecting passwords + safeguarding files + ensuring trust



🔐 Integrated Project Idea:

"Secure Digital Vault with MFA, Access Control, and File Security Check"

📌 Project Overview

A **Secure Digital Vault** that allows users to:

- **Safely store credentials** (Password Manager)
- **Upload and manage sensitive files** (Secure File Storage)
- **Protect accounts with Multi-Factor Authentication**
- **Automatically check uploaded files for security risks (malware, sensitive data leaks, etc.)** before allowing storage.

This creates a **holistic cybersecurity solution** for individuals and organizations.

🛠️ Detailed Solution

1. User Authentication (Login + MFA)

- Users log in with **username + password**.
- Then, an additional MFA step:
  - OTP via email/SMS, or
  - Authenticator app (TOTP) like Google Authenticator.
- Only after MFA, access to vault is granted.

🔑 Tech Options:

- `bcrypt` / `argon2` for password hashing
- `PyOTP` for MFA in Python

2. Password Manager Module

- Encrypted storage for user credentials (AES-256 encryption).
- Each password is encrypted with a **user-specific master key**.
- Users can **add, edit, retrieve** their credentials securely.

🔑 Tech Options:

- Python `cryptography` library (Fernet/AES)
- SQLite database with encrypted fields

3. Secure File Storage with Access Control

- Users can **upload, download, and manage files**.
- Files are:
  - **Encrypted before storage** (AES-256)
  - Tagged with **access policies** (e.g., only owner, shared with group).
- **Audit Trail**: Every upload, download, delete is logged with timestamp + user ID.

🔑 Tech Options:

- File encryption: `cryptography` AES
- Database for metadata: SQLite / PostgreSQL

4. File Security Check (New Feature 🚨)

When a user uploads a file:

1. **Malware Scan**
   - Run file through a malware scanner (e.g., ClamAV open-source antivirus).
   - Block storage if malware detected.

2. **Sensitive Data Check**
   - Use regex/ML to detect if the file contains sensitive info (e.g., credit card numbers, SSNs, passwords).
   - Warn user before storing.

3. **File Type Validation**
   - Allow only safe file types (e.g., `.pdf`, `.docx`, `.txt`).
   - Block risky ones (`.exe`, `.bat`, `.js`).

✅ If the file passes checks → encrypt + store.
❌ If it fails → reject and notify user.

🔑 Tech Options:

- Malware scanning: **ClamAV** (can integrate with Python)
- File type check: MIME type validation (`python-magic`)
- Regex checks for sensitive data (e.g., `\b\d{16}\b` for credit cards)

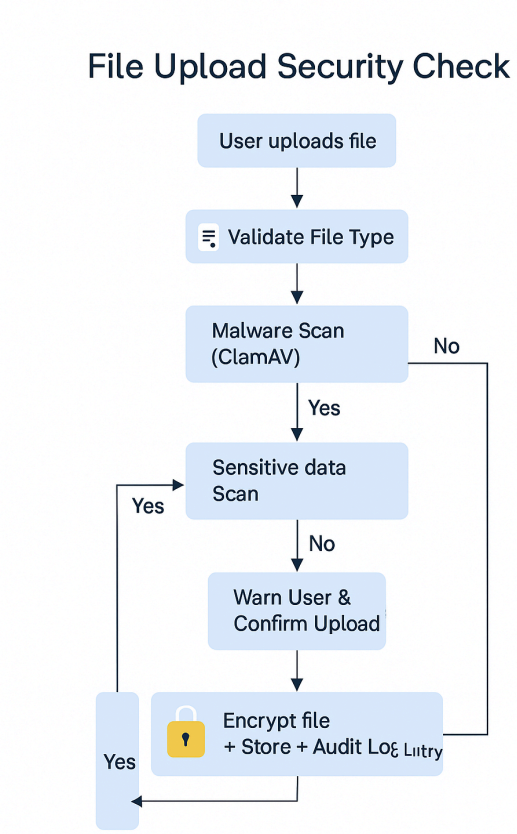
🚀 Implementation Plan

1. **Phase 1**: User Authentication + MFA.
2. **Phase 2**: Password Manager (AES-encrypted credentials).
3. **Phase 3**: Secure File Storage with encryption + access control.
4. **Phase 4**: File Security Check (malware scan + sensitive data scan).
5. **Phase 5**: Audit Trail + Reporting.

High-Level Architecture Diagram

```
┌──────────────────────────┐
│        User Device       │
│ (Web/Mobile App Frontend) │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐
│    Authentication Layer  │
│ (Login + Password + MFA) │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐
│      Application Core    │
│ 1. Password Manager Module │
│ 2. Secure File Storage   │
│ 3. File Security Checker │
│ 4. Access Control & Logging │
└──────────────────────────┘
            │
      ┌─────┴─────┬──────────┐
      ▼           ▼          ▼
┌──────────┐ ┌──────────┐ ┌──────────────┐
│ Credential DB │ │ File Storage │ │ Audit Log DB │
│ (Encrypted) │ │ (Encrypted) │ │ (User Actions) │
└──────────┘ └──────────┘ └──────────────┘
```

Flowchart: File Upload Security Check



🔐 **Example Overview – SecureVault 360 in Action**

Imagine a **university research lab** that deals with highly sensitive data like:

- Student personal records
- Research papers before publication
- Intellectual property (patents, experimental results)

Currently, these are stored in **local drives or shared folders**, making them vulnerable to:

- Unauthorized access
- File tampering
- Credential theft

How SecureVault 360 Helps:

1. **Password Manager with MFA**
    - Each professor, student, and researcher logs in using **multi-factor authentication** (password + OTP/email/mobile).
    - Credentials (for research portals, lab equipment, cloud services) are stored securely in the **encrypted password vault**.

2. **Secure File Storage with Access Control**
    - Research papers and experiment data are uploaded into the **secure vault**.
    - Files are encrypted automatically.
    - Access control ensures that only **authorized team members** can view or edit files.

3. **File Security Check on Upload**
    - When a researcher uploads a file (say, a dataset), the system automatically:
        - Scans it for **malware/viruses**
        - Verifies its **integrity** using hashing
        - Ensures it's stored in an **encrypted format**

✅ **Result**: The university lab is now protected against **credential theft, malware uploads, and unauthorized data leaks**.

Problem Statement

In today's digital era, individuals and organizations handle countless credentials and sensitive files daily.

- Users struggle to remember multiple strong passwords, often reusing weak ones.
- Sensitive files are stored on personal devices or cloud drives without encryption or proper access control.
- Uploaded files may carry hidden malware, risking data leaks and system compromise.

These issues expose users to identity theft, data breaches, and financial loss. There is an urgent need for a unified system that can securely manage credentials and protect sensitive files.

Proposed Solution

We propose **SecureVault 360**, an integrated cybersecurity system that combines:

1. **Password Manager with Multi-Factor Authentication (MFA)**
    - Stores all user credentials in an encrypted vault.
    - Provides stronger login protection using MFA.

2. **Secure File Storage with Access Control**
    - Encrypts uploaded files to prevent unauthorized access.
    - Grants permissions based on user roles.

3. **File Security Check on Upload**
    - Scans uploaded files for malware or suspicious content.
    - Generates integrity hashes (SHA-256) to ensure files are not tampered with.

✅ Together, these features ensure **end-to-end protection of user data and files**.

Future Scope

- **AI-driven Threat Detection**: Integrate machine learning models to detect advanced malware patterns in uploaded files.
- **Cloud Integration**: Expand secure vault storage to cloud platforms (AWS, Azure, GCP) with zero-trust access.
- **Blockchain-based Audit Trails**: Use blockchain to create immutable logs of file uploads, access, and modifications.
- **Biometric Authentication**: Add fingerprint/face recognition for stronger password vault access.

- **Enterprise Deployment**: Scale the system for organizations with centralized admin dashboards and monitoring.